

Templated Functions

C++ templates can be used both for classes and for functions in C++. Templated functions are actually a bit easier to use than templated classes, as the compiler can often deduce the desired type from the function's argument list.

The syntax for declaring a templated function is similar to that for a templated class:

```
template <class type> type func_name(type arg1, ...);
```

For instance, to declare a templated function to add two values together, you could use the following syntax:

```
template <class type> type add(type a, type b)
{
    return a + b;
}
```

Now, when you actually use the add function, you can simply treat it like any other function because the desired type is also the type given for the arguments. This means that upon compiling the code, the compiler will know what type is desired:

```
int x = add(1, 2);
```

will correctly deduce that "type" should be int. This would be the equivalent of saying:

```
int x = add<int>(1, 2);
```

where the template is explicitly instantiated by giving the type as a template parameter.

On the other hand, type inference of this sort isn't always possible because it's not always feasible to guess the desired types from the arguments to the function. For instance, if you wanted a function that performed some kind of cast on the arguments, you might have a template with multiple parameters:

```
template <class type1, class type2> type2 cast(type1 x)
{
    return (type2)x;
}
```

Using this function without specifying the correct type for type2 would be impossible. On the other hand, it is possible to take advantage of some type inference if the template parameters are correctly ordered. In particular, if the first argument must be specified and the second deduced, it is only necessary to specify the first, and the second parameter can be deduced.

For instance, given the following declaration

```
template <class rettype, class argtype> rettype cast(argtype x)
{
    return (rettype)x;
}
```

this function call specifies everything that is necessary to allow the compiler deduce the correct type:

```
cast<double>(10);
```

which will cast an int to a double. Note that arguments to be deduced must always follow arguments to be specified. (This is similar to the way that default arguments to functions work.)

You might wonder why you cannot use type inference for classes in C++. The problem is that it would be a much more complex process with classes, especially as constructors may have multiple versions that take different numbers of parameters, and not all of the necessary template parameters may be used in any given constructor.

Templated Classes with Templated Functions

It is also possible to have a templated class that has a member function that is itself a template, separate from the class template. For instance,

```
template <class type> class TClass
{
    // constructors, etc

    template <class type2> type2 myFunc(type2 arg);
};
```

The function myFunc is a templated function inside of a templated class, and when you actually define the function, you must respect this by using the template keyword twice:

```
template <class type> // For the class
    template <class type2> // For the function
    type2 TClass<type>::myFunc(type2 arg)
    {
        // code
    }
```

The following attempt to combine the two is **wrong** and will not work:

The following attempt to combine the two is **wrong** and will not work.

```
// bad code!  
template <class type, class type2> type2 TClass<type>::myFunc(type2 arg)  
{  
    // ...  
}
```

because it suggests that the template is entirely the class template and not a function template at all.